

ENTITLEMENT-AWARE RAG • COMPANION QUIZ DECK

Thirteen Questions

One or more per module. Every question is answerable from the main deck — and every wrong answer is a real production failure someone has shipped.

13 questions • 26 slides • answers included

Run alongside the 3-hour course, or standalone as a 45-minute review session

Thirteen questions, eight modules, no concept left unexamined.

Module 1 • The problem	Q1 relevance vs authorization • Q2 existence leakage
Module 2 • Access models	Q3 choosing a model • Q4 permissions through chunking
Module 3 • Mathematics	Q5 post-filter recall • Q6 hard constraint • Q7 authorized recall
Module 4 • Architecture	Q8 entitlement freshness • Q9 cache keys
Module 5 • Evaluation	Q10 URR vs UCE • Q11 the eval record • Q12 release gates
Modules 6–8 • Ship it	Q13 vendor selection and fail-closed behaviour

Module 5 carries three questions because it carries the course: design is judged by whether it can be proved.

An employee asks about Project Phoenix. The top semantic match is an executive acquisition memo. What went wrong?

“What are the company’s plans for Project Phoenix?”

- A** The retriever malfunctioned — it should have ranked the memo lower
- B** The embedding model needs fine-tuning on access-control signals
- C** Nothing went wrong in retrieval; the corpus should not contain the memo
- D** Nothing went wrong in retrieval — and that is precisely the problem

Pick one. Then answer the harder question: does improving your retriever make this better or worse?

D — the retriever did its job perfectly, and a perfect retriever over a mixed corpus is an efficient leak.

Improving relevance without authorization increases risk.

Why not A

the memo genuinely is the best semantic answer to that question. Ranking it lower would mean building a worse retriever on purpose, and it would fail the moment someone phrases the query differently

Why not B

authorization is a property of the (user, document) pair evaluated at request time. An embedding encodes meaning, not entitlement — there is no vector for “Alice may read this”

Why not C

removing sensitive content from the corpus destroys the value of enterprise RAG, and “sensitive” is not a property of the document — it is a relation between the document and each user

The correction

the search space itself must depend on identity: $D_u = \{ d \text{ in } D : \text{Authorized}(u,d) = 1 \}$, then rank within D_u

The failure mode is inverted from what security teams expect: the better your embeddings, the more precisely you surface the most sensitive document.

This refusal is correct, polite, and a security incident. What leaked?

“I found Acquisition-of-Company-X.pdf, but you don’t have access to it.”

- (a) What information crossed a boundary, given that no document content was disclosed?
- (b) Which of the six leakage surfaces from Module 1.5 does this belong to?
- (c) What kind of denial was actually required — and what does it cost the user experience?

Then: name two other places in a RAG product where this same leak occurs by default.

The filename leaked a material corporate event — and no content was disclosed at all.

Content denial is not existence denial. Your tests must distinguish the two.

What crossed	the existence of an acquisition, the counterparty’s name, and the fact that documentation already exists — enough to act on, and in a public company potentially a securities-disclosure problem
Which surface	citation and metadata leakage — surface four of six, and the one teams most often have no control for at all
What was required	resource-discovery denial: the user must not learn the document exists. The cost is real — “no results found” is less helpful than “you lack access”
Where else it hides	result counts (“3 of 47”), autocomplete and did-you-mean suggestions, and differing error codes for forbidden versus not-found

A system that denies content while confirming existence is still leaking — and it will pass every content-based test you write.

Alice and David are both engineers. Only Alice works on Project Falcon. Which model separates them cleanly?

Query: “What is the budget for Project Falcon?” Expected: Alice allowed · David denied

- A** RBAC — give David a different role
- B** ACL — list Alice on every Falcon document
- C** ReBAC — permission follows the membership path Alice → Engineering → Falcon → document
- D** None of these; this requires a fine-tuned model

Then: what specifically goes wrong if you answer A? And what goes wrong at scale if you answer B?

C — and the reason is that project membership is a relationship, not a job title.

RBAC produces role explosion. ACL produces list explosion. ReBAC follows the path.

What goes wrong with A

you mint Engineer-Falcon, then Engineer-Falcon-EMEA, then Engineer-Falcon-EMEA-Contractor. Within a year nobody can say what a role grants, and joiner-mover-leaver hygiene silently breaks

What goes wrong with B

correct, but every Falcon document carries an explicit list, every membership change rewrites many documents, and a user in thousands of groups produces a query filter with thousands of terms

Why C fits

enterprise permission is inherited and transitive — folders, projects, org hierarchy, delegation. ReBAC expresses that natively; OpenFGA and SpiceDB answer “can u read d” in milliseconds

The honest answer

ABAC with a project attribute also works. Production systems layer all four: RBAC for coarse gates, ABAC for classification and residency, ReBAC or ACL for specifics — normalized into one descriptor

The trap in the question is that all three of A, B and C are correct today for two users. Only one of them is still correct at 50,000.

A team ships content ingestion this sprint and permission ingestion “next sprint.” What is now true of the index?

documents → parse → chunk → embed → index (ACLs: TODO)

- (a) What is the state of every chunk in the index right now?
- (b) What is the single most likely shortcut a developer takes to make search “work” in the meantime?
- (c) What is the one-sentence test that would have caught this before launch?

Bonus: name three derived artefacts that must inherit the source document’s permissions.

Content survives chunking effortlessly. Permissions do not — and a missing descriptor usually becomes “public.”

Every chunk must be independently authorizable. The retrieval unit is the chunk, so the security unit must be the chunk.

The state of the index

chunks with perfect embeddings and no security descriptor. A filter of the form “allow if groups match” now matches nothing — or, if written defensively, matches everything

The shortcut

treating absent permissions as unrestricted, so that search returns results and the demo works. This is the single most common origin of a real production leak

The test

pick ten chunks at random from your index and ask: if this chunk were retrieved on its own, could the system say who is allowed to see it?

Derived artefacts

summaries · embeddings of summaries · extracted entities and graph nodes · cached rewrites · anything copied into traces and analytics. A summary of a confidential document is confidential

And note the re-index hazard: changing chunk size rebuilds the corpus, which is a fresh opportunity to lose every descriptor you carefully attached.

K equals ten. Only three of the ten candidates are authorized. What does the user receive, and why is this worse than it looks?

Retrieve top 10 ignoring authorization → remove unauthorized → ?

- (a) How many results does the user get — and is that because only three relevant authorized documents exist?
- (b) Which population of users does this failure mode hurt most, and why?
- (c) Name three mitigations, and say which one is the real fix rather than a workaround

Then the security question: even though nothing leaked, what is weaker about this design than pre-filtering?

Three — and not because only three exist. The user asked for the ten best answers and got three.

Post-filtering degrades worst for exactly the users with the narrowest entitlements.

Why it looks fine to you

there may be dozens of authorized relevant documents one page deeper. The system looks broken to the user and healthy to the operator — no error, no alert, just thin answers

Who it hurts

contractors, new joiners, users outside the primary region — the narrow-entitlement majority. Executives, who often evaluate the tool, see none of it because almost everything is authorized for them

Mitigations

oversampling · adaptive K · iterative retrieval · index partitioning · hybrid pre/post — and filtered ANN, which is the real fix because it removes the cause rather than buying back the symptom

The security gap

unauthorized documents were retrieved, scored, and existed inside your process — in memory, in traces, possibly in a candidate cache. Pre-filtering shrinks the whole threat surface, not just retrieval

Post-filtering is defensible as a second gate on top of pre-filtering. It is rarely defensible as the only gate on a mixed-sensitivity corpus.

A colleague proposes modelling authorization as a ranking feature. Give three reasons to reject it.

$$\text{Score} = \text{Relevance} + 0.2 \cdot \text{Permission}$$

- (a) What can an attacker do with a sufficiently well-crafted query?
- (b) What happens to this weight the first time someone tunes the system for NDCG?
- (c) What can you no longer assert in CI?

Then state the correct formulation in one line.

It is exploitable, unauditable, and untestable — a security bug with a decimal point.

$$A(u, d) = 0 \Rightarrow d \text{ is not in the candidate set.}$$

Exploitable

with a soft weight, a sufficiently relevant unauthorized document outranks a weakly relevant authorized one. The attacker's job reduces to writing a very good query

Unauditable

"why did Alice see this document?" must have a deterministic answer — a policy id and a version. A weighted sum is not an answer a CISO can accept or a regulator can review

Untestable

hard constraints give you an invariant to assert on every build: for every retrieved d , $A(u, d) = 1$. A weighted sum gives you a distribution of scores and nothing to assert

And it will drift

the weight is a hyperparameter. The first tuning run that raises relevance and lowers permission will improve NDCG and look like a win on every dashboard you have

The untestable point is the one that matters most: this invariant becomes Layer 2 of the seven-layer proof in Module 5.

Four authorized relevant documents exist for this user. Three were retrieved. Compute Authorized Recall@K — then explain the denominator.

$$\text{AuthorizedRecall@K} = \text{relevant authorized documents retrieved} / \text{relevant authorized documents available}$$

- (a) What is the value?
- (b) Why is the denominator not “all relevant documents in the corpus”?
- (c) What would happen to your engineering incentives if you used the wrong denominator?

Then: why must this metric be reported per persona rather than in aggregate?

0.75 — and the denominator is what makes it a security-aware metric rather than an ordinary one.

3 / 4 = 0.75. A document the user cannot access is not a miss; it is correctly absent.

Why the denominator is restricted

scoring an unauthorized document as a miss punishes the system for obeying policy — the metric would rate a correctly-secured system as a poor one

The incentive failure

point an optimisation loop at the wrong denominator and you are paying engineers to weaken the filter, because that is the only way to move the number

What this implies for your data

gold data must label relevance and authorization separately, per user — which is exactly the three-list eval record in Module 5.1

Why per persona

aggregate authorized recall is dominated by broad-entitlement personas. A catastrophic contractor experience hides comfortably inside a healthy average

Report Authorized Recall@K and Authorized Precision@K together, at the K you actually ship, broken down by persona.

Where exactly is the leak in this timeline — and what is the name of the quantity you should be measuring?

```
10:00 Alice belongs to Project-X
10:01 Project document indexed with Alice authorized
10:05 Alice removed from Project-X
10:06 Alice asks about Project-X
```

- (a) Is the filter broken? Is the code wrong?
- (b) Which component learns about the revocation first, and which learns last?
- (c) What single architectural change bounds this leak regardless of sync speed?

Then: what is your own system's entitlement lag right now — and who knows it?

Nothing is broken. The filter is correct, the code is correct, and the data is five minutes old.

$$\text{EntitlementLag} = T_{\text{index_updated}} - T_{\text{permission_changed}}$$

Why it is insidious

a correct filter over stale permissions is still a leak — and every security test you have written passes, because the defect is in freshness rather than logic

Who knows first, who last

the identity provider or HR system knows immediately; the search index knows at the next sync. That ordering is structural, so index-only designs have a revocation floor equal to their sync interval

The bounding change

resolve entitlements live at the authorization recheck, immediately before context construction — the index filter generates candidates, the live check decides

What to do Monday

measure the lag. If it is not measured it is not bounded, and an unbounded staleness window is an unbounded leak window

This is why mature systems do both: index-side filtering for candidate generation, live revalidation before the prompt.

Your semantic cache is keyed on the question. Alice the executive asks; Bob in marketing asks the same thing an hour later. Trace what happens.

```
cache_key = hash( normalized_question )
```

- (a) Which security controls run on Bob's request?
- (b) Why is this worse than an ordinary authorization bug?
- (c) Write a safer cache key — and then name the harder problem the key does not solve

Bonus: even with a correct key, how can a cache still leak?

None of your security controls run. A cache hit skips retrieval, so it skips the filter, the recheck, and every gate.

hash(normalized_query + tenant + entitlement_context + policy_version)

Why it is worse than a bug

the controls are all present and correct — they simply never execute. Tests that exercise the retrieval path cannot see it, because the failing request never enters that path

What the safer key buys

isolation. Two users with different entitlements can no longer collide, and a policy change invalidates the old namespace

What it does not solve

invalidation. A user's entitlements can shrink after an entry was created for them — isolation without invalidation still serves yesterday's privileges

The side channel

even with a correct key, a 40-millisecond response where 2 seconds is normal tells Bob that someone recently asked this and an answer exists

Make cross-user cache leakage a mandatory regression test — and test with a paraphrase, because the whole point of a semantic cache is that near-miss questions hit the same entry.

Your dashboard shows Unauthorized Retrieval Rate at zero and Unauthorized Context Exposure above zero. Diagnose it.

$$URR = 0 \quad UCE > 0$$

- (a) Is this possible? If retrieval never returned an unauthorized chunk, how did one reach the prompt?
- (b) Name three concrete paths that produce this signature
- (c) Which of the three combinations — $URR > 0 / UCE = 0$, $URR > 0 / UCE > 0$, $URR = 0 / UCE > 0$ — is an incident, and which is merely untidy?

Then: what field must your telemetry event carry for you to compute these two separately at all?

Something is writing into the context without passing through retrieval — and that is the most alarming of the three combinations.

Retrieval never saw it. The gate never ran. The content is in the prompt anyway.

The three paths

a semantic cache hit that skipped retrieval entirely · conversation or session memory carrying an earlier turn's content · an agent scratchpad or tool result written straight into the prompt

URR > 0, UCE = 0

untidy but benign — the retriever reached, the gate held. Tune retrieval when convenient; no incident

URR > 0, UCE > 0

a gate failure and a genuine incident: the content reached the model, therefore the inference provider, therefore your request logs and traces

URR = 0, UCE > 0

an architectural bypass. Investigate urgently — an entire code path is reaching the prompt without traversing your security pipeline

And the telemetry field that makes any of this computable is `entered_llm_context` — one boolean per chunk per request.

Here is a conventional RAG eval record. What single change makes leak detection possible — and what are the three document lists?

```
{ "query": "What is the vacation policy?", "expected_documents": ["HR-102"] }
```

- (a) Why can this record never detect an authorization failure, no matter how many cases you write?
- (b) What must be added, and what does ExpectedBehavior contain that ExpectedAnswer does not?
- (c) Name the three document lists and say which one is the actual test

Then: where should the authorized list come from — a tester's judgement, or somewhere else?

Add identity. Without a user in the record, no number of test cases can detect a leak.

relevant · authorized · unauthorized — and the unauthorized list is the test.

Why the old record cannot work

it has no user, so it cannot express “Alice may see this and Bob may not.” A suite that cannot express the property cannot test it, however large it grows

What ExpectedBehavior holds

a behaviour class, not a string: answer_from_authorized_sources_only · deny_content · deny_existence · partial_answer_with_declared_gaps · escalate_to_human

Why three lists

relevance and authorization are independent labels, and the interesting cases are exactly where they disagree — a document that is highly relevant and forbidden

Where the authorized list comes from

derive it mechanically from the policy engine, not from a tester’s judgement, so the eval encodes intended policy. Verify the policy itself separately — that is Layer 1 of the proof

If falcon-budget appears in retrieval, in context, in the answer, or in a citation, the case fails — no matter how good the answer was.

The release candidate improves NDCG by three points and fails one cross-user cache test. Ship or hold?

$$URR = 0 \cdot UCE = 0 \cdot ALR = 0 \cdot \text{Cross-user cache leakage} = 1 \cdot NDCG + 3.0$$

- (a) Ship or hold — and on what basis, given that three of the four security metrics are clean?
- (b) What would a weighted “system quality score” have concluded here?
- (c) Why is this a boolean gate rather than a threshold?

Then: what is the one sentence you send to the team that wanted to ship?

Hold. One cross-user cache leak means one user received another user's confidential answer.

CrossUserLeak > 0 ⇒ FAIL. Not “ships with a caveat.” Not “ships behind a flag.”

Why three clean metrics do not help

the gates are independent conditions, not a portfolio. Passing three of four is not a passing grade; it is one confirmed leak with good company

What a blended score would say

three points of NDCG against one failed cache test reads as a net improvement. That is precisely how a leak disappears into a weighted mean

Why boolean, not threshold

a threshold invites negotiation, and negotiation under launch pressure moves in one direction. Exposure is an equality constraint at zero — it is a condition you satisfy, not a quantity you trade

The anticipated objection

“this will slow us down.” It will not: these are automated tests that run in CI in minutes. What slows teams down is discovering three weeks before launch that the architecture cannot support authorization

Only after the security gates pass should teams optimise relevance, latency, and cost.

A vendor answers “yes, we support ACLs.” What have you learned — and what should you have asked instead?

“Which vector database supports ACLs?”

- (a) Why does every vendor pass this question, and what does passing it prove?
- (b) State the two-part question that actually discriminates between platforms
- (c) Their policy engine becomes unreachable under load. What are the three possible behaviours, and which is correct?

Then: which single question would you send in writing before agreeing to a demo?

You have learned that a filter syntax exists. Ask instead where the authoritative decision is made — and how it is enforced at every boundary.

Where is the authoritative authorization decision made, and how is it enforced at every retrieval boundary?

Why the first question fails	metadata filtering is table stakes, so every vendor passes. It tells you nothing about the source of truth, freshness, selective-filter performance, or failure behaviour
What the second question forces	a named source of truth, a named decision point, and a named enforcement mechanism at each of five boundaries: candidates, rerank, context, citation, cache
Fail-closed, ranked	serve unfiltered = catastrophic · serve nothing = correct but blunt · serve from a bounded-age cached entitlement set = usually the right engineering answer. Insist it is documented and demonstrated
The written question to send first	“Show recall and latency at one percent filter selectivity, on an index of our size.” Filtered ANN is where marketing and engineering separate, and it cannot be answered with a syntax example

A platform that fails open under load is a platform that leaks under load — and a platform that cannot be continuously tested cannot be continuously trusted.

Thirteen answers, for the handout.

- Q1** The retriever worked perfectly — that is the problem. Relevance does not imply authorization
- Q2** The filename leaked a corporate event. Content denial is not existence denial
- Q3** ReBAC (or ABAC by project). RBAC explodes into roles; ACL explodes into lists
- Q4** Chunks with no descriptor, and “missing means public.” Every chunk must be independently authorizable
- Q5** Three results, not because three exist. Post-filtering hurts narrow-entitlement users most
- Q6** Exploitable, unauditably, untestable. $A(u,d) = 0$ removes d from the candidate set
- Q7** 0.75. The denominator is the authorized corpus, or you incentivise weakening the filter
- Q8** Nothing is broken; the data is five minutes old. Measure EntitlementLag; revalidate live
- Q9** None of your controls run. Key on tenant + entitlement context + policy version — and invalidate
- Q10** An architectural bypass: cache, session memory, or agent scratchpad writing into the prompt
- Q11** Add identity. relevant / authorized / unauthorized — the unauthorized list is the test
- Q12** Hold. CrossUserLeak $> 0 \Rightarrow$ FAIL, regardless of the NDCG gain
- Q13** Only that a filter syntax exists. Ask where the decision is made — and test fail-closed by experiment

END OF QUIZ

**Every wrong answer in this deck
is a system someone has shipped.**

Secure RAG = Relevant Retrieval + Deterministic Authorization + Continuous Verification

The questions are not hypothetical. Each one describes a production failure that passed somebody's review.